HERMES · PHASE 0 · LESSON 0005 · LAB EXERCISE 04-VALIDATE-THE-BOUNDARY · ~45–60 MIN

# Validate the Boundary

*Every response type in this course so far has been a claim. When the bytes stop matching the types, which layer notices?*

> **Why this matters for Hermes**
> Scenario step S1 (the job envelope is parsed) rejects invalid work before a token is spent. Step S2 (evidence is checked) runs the same parse over a Context Pack. Today you build that parse. It pays the boundary rule opened as debt in lesson 0001, and it closes Phase 0.

## 1 · The lie that travels

Every response in this course has taken one path. Bytes arrive, `JSON.parse` checks the syntax, and an assertion attaches a type.

A *JSON boundary* is the seam where serialized bytes enter your typed code. The `as MessageResponse` of exercise 01 and the SDK's declared `Message` return type are the same move at that seam. Both are claims the compiler believes and the runtime never checks, because TypeScript erases types.

The mock can now make the claim false. Send it the header `x-mock-scenario: drift`.

```
POST /v1/messages
x-api-key: … · anthropic-version: 2023-06-01 · x-mock-scenario: drift

HTTP/1.1 200 OK                        ← a healthy status
content-type: application/json         ← the same media type
request-id: req_mock_0001              ← the same headers

{
  "id": "msg_mock_0001",
  "type": "message",
  …
  "stop_reason": "end-turn",           ← the API allows four words. This is not one of them.
  "usage": {
    "input_tokens": 11,
    "output_tokens": "42",             ← the API sends a number here
    …
  }
}
```

Part A is exercise 01's pipeline, unchanged. Run it against that reply and it prints:

```
typeof output_tokens (TS says number): string
ledger — tokens spent this run: 1142        ← 11 + "42" joined as strings
typeof spent (TS says number): string
over the 1000-token ceiling? true           ← the run spent 53
```

No line threw. JavaScript's `+` met a string and joined the two values. The comparison turned `"1142"` back into a number, so a budget check fired that should never fire. Other digits run the fault the other way, and a real overrun passes.

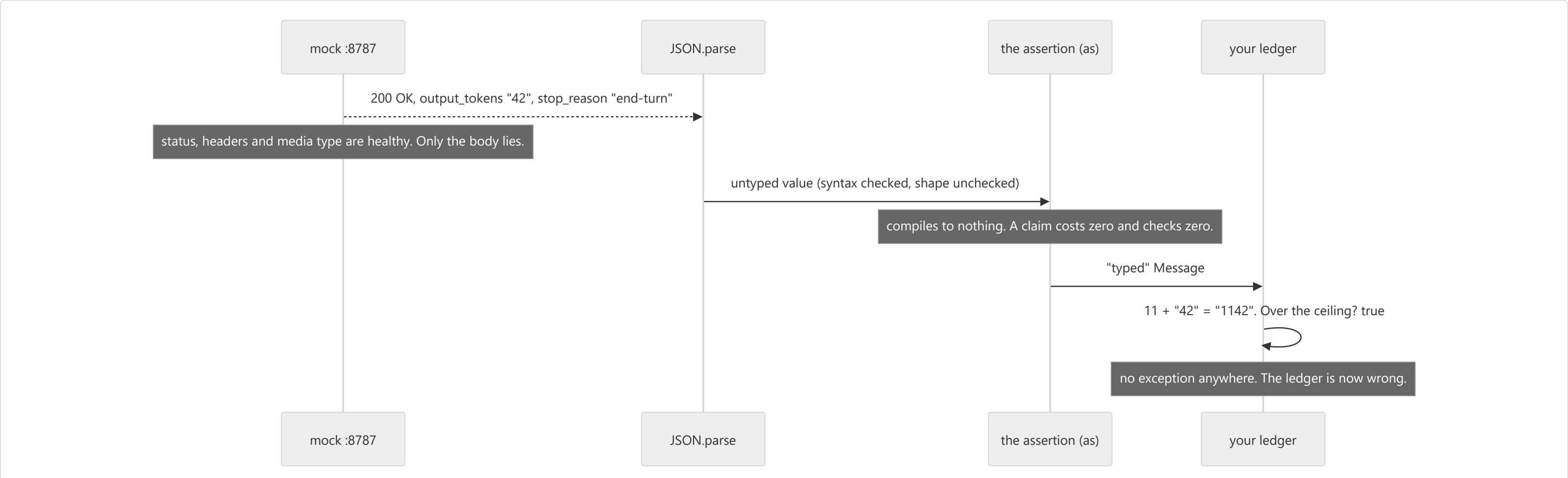
mock :8787
JSON.parse
the assertion (as)
your ledger
200 OK, output_tokens "42", stop_reason "end-turn"
status, headers and media type are healthy. Only the body lies.
untyped value (syntax checked, shape unchecked)
compiles to nothing. A claim costs zero and checks zero.
"typed" Message
11 + "42" = "1142". Over the ceiling? true
no exception anywhere. The ledger is now wrong.

Diagram 1. Part A, measured. The run spent 53 tokens and the ledger claims 1142.

| Layer | What it saw |
|---|---|
| the six responsibilities ①–⑥ | nothing moved. They govern the request, and this lie rides in a successful body |
| HTTP status and headers | nothing wrong. HTTP vouches for the exchange, never for the body's shape |
| the TypeScript compiler | nothing. It finished before the bytes existed, and its types are erased |
| `as MessageResponse`, and the SDK's `Message` | both still succeed. An assertion compiles to zero code |
| your running code | everything. It is the only layer left, and it meets the lie as a wrong number |

> **New today.**
> Five ideas are new.
>
> - A schema is a runtime value that encodes a type.
> - A check at the boundary replaces the claim.
> - A failed parse is data, with a path per problem.
> - `z.infer` derives the static type from the schema.
> - One check at the edge is enough. Typed code carries the proof inward.
>
> Reprise: type erasure from lesson 0001, narrowing a union from 0004, and the mock from 0002.

## 2 · The schema is a value that runs

Zod supplies the runtime half. Read its entry point as an object path, the way lesson 0003 read the SDK's.

```
import { z } from "zod";          // z — a namespace OBJECT. It survives compilation.

const UsageSchema = z.object({    // property access + call → a ZodObject INSTANCE
  input_tokens: z.number().int().nonnegative(),
  output_tokens: z.number().int().nonnegative(),
});

const result = MessageSchema.safeParse(body);   // a method on that instance, fed `unknown`
```

| Syntax | Meaning | What exists at runtime |
|---|---|---|
| `z` | a named import from `"zod"` | an object of factory functions, where an `interface` leaves nothing. Measured: `typeof z === "object"` |
| `z.object({ … })` | call a factory with a field map | an instance holding your field schemas as data. Measured: `constructor.name === "ZodObject"` |
| `.safeParse` | method lookup on that instance | a function bound to the schema value |
| `safeParse(body)` | invoke it with unchecked input | code that walks the bytes and collects every disagreement |
| `z.infer<typeof MessageSchema>` | a type-level query, in §4 | nothing. It is erased, and the schema stays |

A *Zod schema* is a runtime value that encodes a shape. A signature tells TypeScript what may be called. The implementation does the runtime work. Exercise 01's interface was a signature with no implementation anywhere, and Zod is the implementation the boundary was missing.

You have written one of these by hand already. The mock's gate ④ narrows `unknown` with a page of `if` checks. A schema is that page, declared once and composable.

One word now names three operations.

| The phrase | What it checks | What you get | On failure |
|---|---|---|---|
| `JSON.parse(text)` | JSON syntax | an untyped structure, best typed as `unknown` | throws `SyntaxError` |
| `MessageSchema.parse(data)` | the declared shape, at runtime | the validated, typed value | throws `ZodError` |
| `MessageSchema.safeParse(data)` | the declared shape, at runtime | a union you narrow, in §3 | returns `{ success: false, error }` and throws nothing |
| the pipeline of lessons 0001 to 0004 | syntax, then nothing | a claim wearing a type | silence, as §1 measured |

A Zod schema is not JSON Schema, which is a serialized description that travels between programs. Phase 2's MCP tools declare their inputs with it.

## 3 · The rejection is data

`parse` and `safeParse` are two methods, not one overload. Lesson 0004's `create()` changed its return type when you passed `stream: true`. Here you choose by name.

```
// zod/v4/classic/schemas.d.cts, abridged
parse(data: unknown, …):     core.output<this>;                      // the value, or it throws
safeParse(data: unknown, …): parse.ZodSafeParseResult<core.output<this>>;

// zod/v4/classic/parse.d.cts, lines 3 to 13, each member folded onto one line:
type ZodSafeParseResult<T> = ZodSafeParseSuccess<T> | ZodSafeParseError<T>;
type ZodSafeParseSuccess<T> = { success: true;  data: T;       error?: never };
type ZodSafeParseError<T>   = { success: false; data?: never;  error: ZodError<T> };
```

A *safe parse* is a check that returns its verdict instead of throwing. You narrow the result on `result.success`, the same move as `block.type` in lesson 0001 and `event.type` in lesson 0004. This is your third such union, and the first whose two arms mean proof and refusal. The lab produced these two values:

```
// clean wire — Part C                    // drifted wire — Part B
{                                         {
  success: true,                            success: false,
  data: {                                   error: ZodError {
    id: "msg_mock_…",                      issues: [
    stop_reason: "end_turn",                    { code: "invalid_value",
    usage: {                                      values: ["end_turn", "max_tokens",
      input_tokens: 11,                                   "stop_sequence", "tool_use"],
      output_tokens: 42,                          path: ["stop_reason"],
    },                                            message: 'Invalid option: expected one …' },
    …                                           { code: "invalid_type",
  },                                              expected: "number",
}                                                 path: ["usage", "output_tokens"],
                                                  message: 'Invalid input: expected number,
                                                            received string' },
                                              ],
                                            },
                                          }
```

- One parse reported every disagreement. Zod collected both lies and did not stop at the first.
- Each issue carries a `path`. `["usage", "output_tokens"]` points into the value, so a refusal names the field and the reason.
- The schema holds rules the old type could not state. Exercise 01's interface said `stop_reason: string`, and `"end-turn"` is a string.

`z.enum([…])` lists the four words the API allows, so the schema caught a lie the type system could not express. For a human reader, `z.prettifyError(result.error)` renders the same issues as ✕ lines.

Each method has its place. The throwing one suits reading your own config at startup, where invalid means stop the program. Use `safeParse` where invalid input is an expected outcome. Hermes must refuse the job, record why, and keep supervising.

## 4 · The type comes from the schema

Until today the type came first and the bytes were asserted to match. Two artifacts existed, a hand-written interface and the actual reply, with nothing holding them together. §1 measured the gap.

```
const MessageSchema = z.object({ … });               // the RUNTIME artifact — written once

type Message = z.infer<typeof MessageSchema>;        // the compile-time view — DERIVED
// hover it: { id: string; type: "message"; …; usage: { input_tokens: number; … } }
```

*Schema inference* derives the compile-time type from the schema value. TypeScript reads `typeof MessageSchema` as a handle on that value. The query then extracts the type a successful parse returns.

Delete exercise 01's interface and nothing is lost. One declaration now carries the check and the type, so the two cannot drift apart. Hermes writes the TaskSpec, tool arguments and evidence the same way from Phase 1 on.

The parse also returns the declared subset. Part C measured 9 keys on the wire and 7 across the boundary. Unrecognized keys do not cross, because `z.object` strips them. `z.strictObject` refuses them instead.

One parse where bytes enter is the whole rule. Past it the erased types carry the proof, and a second check redoes the compiler's work. The schema also stays narrower than the SDK's `Message`, and covers only what this client reads.

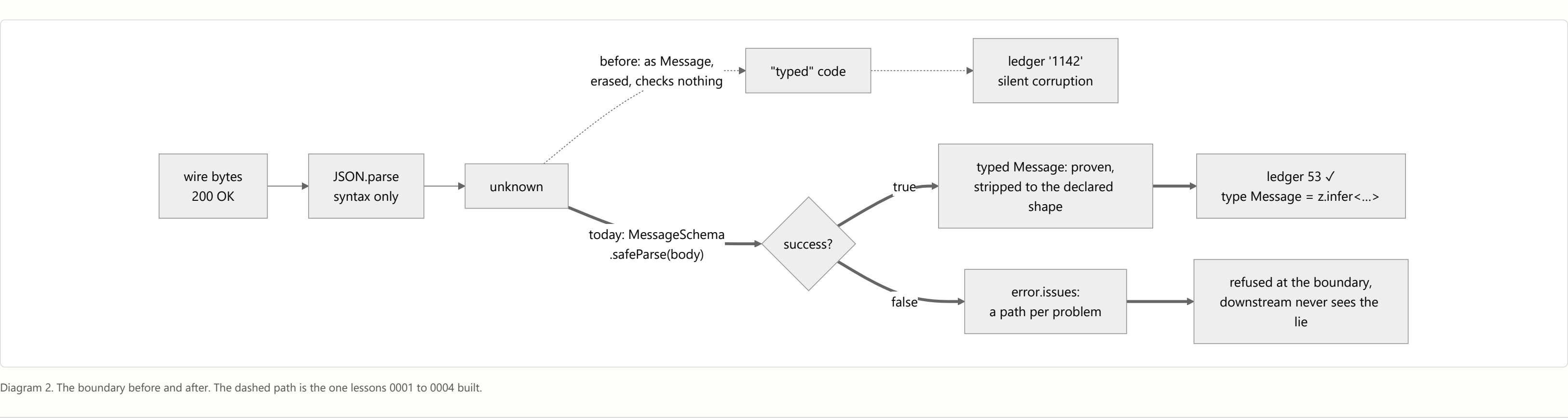

Diagram 2. The boundary before and after. The dashed path is the one lessons 0001 to 0004 built.

| Capability | Status | Where it lives, or when it arrives |
|---|---|---|
| A runtime check on a model reply | implemented | `client.ts`, Parts B and C |
| The type derived from the schema | implemented | `z.infer` in Part C, with the interface deleted |
| A reply that breaks its declared shape | implemented | the mock's `x-mock-scenario: drift` branch |
| The job envelope parsed before dispatch, step S1 | planned | Phase 1's TaskSpec, per `ROADMAP.md` |
| Evidence checked before dispatch, step S2 | planned | Phase 2, per `ROADMAP.md` |
| A refusal recorded in a trace | planned | Phase 1. The issues only print today |

## 5 · The exercise

Commands and the measured table live in `hermes-sdk-lab/04-validate-the-boundary/README.md`. The arc:

1. Run Part A first. It is given, not a task. Watch the ledger go wrong.
2. Open `node_modules/zod/v4/classic/parse.d.cts`. Every claim in §3 is on that page.
3. Part B builds `MessageSchema` and parses the drifted body. Print each issue's path and code, then call `z.prettifyError`.
4. Part C parses a clean reply and recomputes the ledger from `result.data`. Derive `type Message` and compare the key counts.
5. Say which of the six responsibilities changed today. None did.

## 6 · Exercise: who catches it?

### Place each failure with the layer that catches it

Answer from memory before you click. Feedback is immediate.

A typo, `mesages`, in the params object handed to the SDK's `create()`.

the compiler, before the run · an HTTP gate, before any body · the boundary parse, inside a 200 · nothing, the lie travels

A request sent with no `x-api-key` header at all.

the compiler, before the run · an HTTP gate, before any body · the boundary parse, inside a 200 · nothing, the lie travels

`output_tokens` arrives as `"42"`, and the client attaches its type with `as`.

the compiler, before the run · an HTTP gate, before any body · the boundary parse, inside a 200 · nothing, the lie travels

`output_tokens` arrives as `"42"`, and the client parses at the boundary.

the compiler, before the run · an HTTP gate, before any body · the boundary parse, inside a 200 · nothing, the lie travels

`stop_reason` arrives as `"end-turn"`, against a schema whose field is an enum.

the compiler, before the run · an HTTP gate, before any body · the boundary parse, inside a 200 · nothing, the lie travels

A raw `fetch` request body missing `max_tokens`.

the compiler, before the run · an HTTP gate, before any body · the boundary parse, inside a 200 · nothing, the lie travels

Score: 0 / 0

## 7 · Quiz

### Four questions

**Q1. Part A's drifted 200 flowed through `fetch`, `JSON.parse` and `as MessageResponse`. What did the assertion contribute at runtime?**

- It threw the moment output_tokens arrived as a string
- It coerced the drifted string quietly back to a number
- Nothing. It compiled away before the bytes ever existed
- It logged a type warning but allowed the run to continue

**Q2. You call `MessageSchema.safeParse(body)` on untrusted bytes. What comes back?**

- A ZodError thrown for your try/catch to intercept there
- A union you narrow on success, failure riding as data
- A repaired copy in which failing fields are reset to null
- A plain boolean, with any issues printed to the console

**Q3. What does `type Message = z.infer<typeof MessageSchema>` do?**

- Checks the schema against your interface on every build
- Generates a runtime schema from a hand-written interface
- Derives the static type from the schema, one source of truth
- Serializes the schema into JSON Schema for the wire format

**Q4. Where does the runtime check belong?**

- Around every function call, so no value is ever trusted
- Once, where bytes enter. Typed code carries proof onward
- Only inside tests, where fixtures keep the shapes stable
- Inside the SDK, which validates responses before returning

Score: 0 / 0

## 8 · Say it in your own words

Answer these in the chat after the lab, with no notes.

1. Define runtime validation and type erasure. Say why the second makes the first necessary at a JSON boundary.
2. Name what replaced the `as` at the boundary, and say what it is at runtime.
3. Which method does step S1 want? Say why a refusal carrying paths beats an exception.
4. Explain how `11 + "42"` became a budget breach that never happened.

If you keep only four sentences from Phase 0's close, keep these. TypeScript types are erased at runtime, so at a JSON boundary they are claims and not checks. A Zod schema checks what a type only claims, and a safe parse returns its verdict as data. The type comes from the schema, so the claim and the check cannot drift apart. Parse once where bytes enter, then trust the types inside.

## Primary source

Read the Zod page on basic usage, then error customization for where the messages come from. Finish with `node_modules/zod/v4/classic/parse.d.cts`, which is 31 lines you can now read whole.

---

**Version pin.** Every excerpt and number here was verified against `zod` 4.4.3 and `@anthropic-ai/sdk` 0.113.0, by live runs against the exercise 01 mock. Issue codes and error formatting are version-specific. The mock's drift stands in for the real causes: API changes, proxies, stale fixtures and model-generated JSON. The four new terms have notes under `wiki/terms/`.

**Stuck or curious?** Ask your teacher why Zod strips unknown keys by default, or how `z.discriminatedUnion` will type Phase 1's content blocks.

**Phase 0 closes here.** You can trace a full request, read a package's `.d.ts`, and refuse an untrusted byte at the boundary. Next: lesson 0006 wraps the SDK behind a port.